TFG · SONDA PASIVA DE DETECCIÓN PQC

Zeek: qué hace ya y qué falta por hacer

Análisis del analizador pasivo de referencia y delimitación de la aportación del TFG

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **12/10/2026** | Tutoría del miércoles 14/10/2026

1. Qué es Zeek

Un analizador pasivo de tráfico de red de código abierto. No es un IDS de firmas: lo que hace es **reconstruir las conexiones y convertirlas en registros estructurados**. De un fichero pcap o de una interfaz genera una colección de ficheros de log —`conn.log`, `dns.log`, `ssl.log`…— con una línea por conexión y una columna por dato observado.

Internamente lleva reensamblado TCP completo, seguimiento de estado por flujo y un analizador dedicado por protocolo. Encima tiene un lenguaje de scripting propio que permite reaccionar a eventos del handshake.

Se ha instalado la versión **8.0.10 (zeek-lts)** desde el repositorio oficial del proyecto, rama `xUbuntu_26.04`.

2. Lo que Zeek ya hace con TLS

Su `ssl.log` básico registra, por conexión: versión negociada, cipher suite, SNI, si el handshake se completó, la secuencia de mensajes vistos y —lo más relevante aquí— el campo `curve`, que es **el grupo que el servidor escogió para el intercambio de claves**.

Con el script de política `ssl-log-ext`

Zeek incluye `policy/protocols/ssl/ssl-log-ext.zeek`, que amplía el log con los tres campos que esta sonda necesita:

Campo	Qué contiene
`client_curves`	La extensión `supported_groups` (10): la capacidad declarada del cliente
`client_key_share_groups`	Los grupos para los que el cliente envió clave efímera
`server_key_share_group`	El grupo que el servidor escogió

A nivel de scripting, los eventos son directos, con un parámetro `is_client` que distingue las dos direcciones:

```
event ssl_extension_key_share(c: connection, is_client: bool, curves: index_vec);
event ssl_extension_elliptic_curves(c: connection, is_client: bool, curves: index_vec);
event ssl_extension_supported_versions(c: connection, is_client: bool, versions: index_vec);
```

Y su tabla de constantes ya conoce los grupos postcuánticos

Verificado sobre la instalación local:

```
$ grep -i -E "mlkem|kyber" /opt/zeek/share/zeek/base/protocols/ssl/consts.zeek

    # draft-connolly-tls-mlkem-key-agreement-03
    [512] = "MLKEM512",
    [513] = "MLKEM768",
    [514] = "MLKEM1024",
    # draft-kwiatkowski-tls-ecdhe-mlkem-03
    [4587] = "SecP256r1MLKEM768",
    [4588] = "X25519MLKEM768",
    [4589] = "SecP384r1MLKEM1024",
    [25497] = "X25519Kyber768Draft00",
    [25498] = "SecP256r1Kyber768Draft00",
```

3. Resultado sobre la traza de prueba

```
$ zeek -r ../trazas/clienthello_pqc.pcap policy/protocols/ssl/ssl-log-ext
```

Línea única de `ssl.log`, con los campos relevantes:

Campo	Valor	Interpretación
`version`	`TLSv13`	Negoció TLS 1.3
`cipher`	`TLS_AES_128_GCM_SHA256`	Cipher suite elegida
`curve`	**`x25519`**	**Grupo escogido por el servidor**
`client_curves`	`4588,29,23`	X25519MLKEM768, x25519, secp256r1
`client_key_share_groups`	`4588,29`	Solo mandó clave de dos de los tres
`server_key_share_group`	`29`	x25519: clásico
`ssl_client_exts`	`0,10,13,43,21,51`	server_name, supported_groups, sigalgs, supported_versions, padding, key_share
`ssl_history`	`Cs`	ClientHello del cliente, ServerHello del servidor

> **VALIDACIÓN CRUZADA CONSEGUIDA**
>
> Estos valores coinciden **exactamente** con el ClientHello construido por `sonda_tls.py` y con la disección de Wireshark, incluido el orden de las extensiones. La traza de prueba queda validada por **tres implementaciones independientes**.

Dos detalles que se ven en esos datos

- **`client_curves` tiene tres entradas y `client_key_share_groups` solo dos.** Es la distinción entre capacidad declarada y apuesta efectiva, visible en datos reales. Zeek las separa en dos campos precisamente porque son cosas distintas.
- **Los campos extendidos salen como números, no como nombres.** Solo `curve`, del log base, viene traducido. Es decir: incluso usando Zeek hace falta la base de datos de identificadores que pide la reivindicación 8 de la patente.

Nota metodológica: en esta traza el ServerHello procede de la pasarela de salida del entorno donde se generó, no del servidor real. No constituye una medida de ningún servidor concreto; su valor aquí es exclusivamente la validación del analizador.

4. La pregunta que esto plantea

> *Si Zeek ya reensambla TCP, ya parsea el ClientHello y el ServerHello, ya conoce X25519MLKEM768 por su nombre y ya puede registrar `supported_groups`, el `key_share` del cliente y el del servidor… ¿qué aporta este TFG?*

Conviene plantearla abiertamente, porque es la primera objeción que recibirá el trabajo. La respuesta tiene tres partes.

4.1. Zeek describe, no clasifica

Esta es la de más peso. La línea de `ssl.log` reproducida arriba contiene todos los datos del handshake **y ningún veredicto**. No dice si ese servidor es seguro, inseguro o indeterminado. No conoce la diferencia entre un servidor que no soporta postcuántico y uno que sí lo soporta pero evita el HelloRetryRequest.

Toda esa capa de decisión —y la tasa de **45 % de falsos negativos** medida el 12/10/2026 sobre 20 dominios si no se implementa— no existe en Zeek. Es la aportación central de este TFG.

4.2. Zeek no implementa la patente

No construye el inventario con los campos de la reivindicación 10 (rol cliente/servidor, huella del protocolo, tipo de vulnerabilidad, si afecta a claves o a firmas, fecha, modo pasivo o activo), no mantiene estado persistente por servidor entre conexiones, no distingue las dos categorías de inseguridad y no genera alertas. Es una herramienta genérica que casualmente expone algunos de los datos necesarios.

4.3. El enunciado pide una sonda en C

Zeek es un framework extenso en C++ con su propio intérprete de scripts. Una sonda dedicada en C con libpcap es considerablemente más ligera, lo que importa si el destino es un TAP o hardware modesto. Y, en un trabajo de fin de grado, construir el analizador **es** el objeto del trabajo.

5. Lo que a Zeek le falta

Carencia	Comprobación y consecuencia
No conoce los codepoints privados `0xFE30` y `0xFE31`	Verificado: el `grep` sobre `consts.zeek` no devuelve nada. Son los que Cloudflare sigue publicando para los Kyber draft. Si aparecen en una traza, Zeek los muestra como número sin nombre.
Sus constantes citan el Internet-Draft, no la RFC	El comentario de la tabla dice `draft-kwiatkowski-tls-ecdhe-mlkem-03`, cuando la RFC 10024 se publicó en agosto de 2026. Los codepoints coinciden porque IANA los mantuvo, pero la tabla va por detrás del estándar.
El `ssl.log` base **no trae los campos extendidos**	Hay que cargar explícitamente el script de política. Sin él solo se dispone de `curve`.
JA3 / JA4 **no están en el núcleo**	Van en paquete aparte. La huella del protocolo es un campo del inventario según la reivindicación 10.

> **DECISIÓN DE DISEÑO QUE REFUERZA**
>
> Las dos primeras carencias apuntan a lo mismo: **la tabla de identificadores no puede estar compilada en el binario**. Debe cargarse de un fichero externo actualizable, exactamente como pide la reivindicación 8 de la patente. Zeek sirve aquí de ejemplo de lo que ocurre cuando no se hace así.

6. Papel de Zeek en el TFG

La conclusión no es descartarlo, sino reubicarlo: de competidor pasa a instrumento.

Uso	En qué consiste
Verdad de referencia	Pasar la misma traza por Zeek y por la sonda y comparar campo a campo. Da un criterio objetivo de corrección. Es lo que hizo Celia Pascual con Tshark en su TFG.
Línea base de rendimiento	Medir paquetes por segundo de ambos sobre la misma traza. Resultado cuantitativo para el capítulo de evaluación.
Referencia de diseño	Su motor de reensamblado y su analizador de TLS son código abierto y están bien resueltos.

Un contraste ilustrativo: sobre la misma traza, la sonda en C reporta **6 paquetes** y Zeek reporta **1 conexión**. Son dos niveles de abstracción, y pasar del primero al segundo es exactamente el trabajo pendiente: la tabla de flujos y el reensamblado.

7. Resumen para la tutoría

Punto	Evidencia
Zeek cubre la parte de extracción de datos	Reensamblado TCP, parseo de hellos, y con `ssl-log-ext` los tres campos clave
Su tabla ya tiene los grupos híbridos por nombre	4587, 4588, 4589 verificados con `grep` sobre la instalación
Pero no clasifica	La línea de `ssl.log` no contiene ningún veredicto
Esa capa es la aportación del TFG	45 % de falsos negativos medidos si no se implementa
Le faltan codepoints y su tabla cita el draft	Verificado: `0xFE30`/`0xFE31` ausentes; comentario con `draft-…-03`
Se usará como validación y línea base	Traza ya validada contra tres implementaciones independientes